

P1 Final Report

Adapting SCMVC for Multi-View Time-Series Clustering

Self-Weighted Contrastive Fusion for Deep Multi view Clustering with Temporal Encoders

Student	Fahad Ali
Program	MSc Computer Science, University of Vienna
Supervisor	Yllka Velaj
Project	P1 Praktikum
Date	30th June 2026
Repository	SCMVC plus temporal_scmvc extensions

Abstract

This document summarizes the complete P1 work understanding and reproducing SCMVC, adapting it to multi-view time-series datasets, implementing temporal encoders, testing local contrastive learning, and explaining when TCN or LSTM is better based on the observed signal patterns.

1. Executive Summary

The P1 project started from the SCMVC paper and codebase, then moved toward adapting the method for multi-view time-series clustering. The main technical change is to replace the original per-view vector encoder with temporal encoders that process each view as a sequence. I implemented a general TCN version and a general LSTM version, both compatible with HAR, WISDM, MHEALTH, and PAMAP2.

The final standardized experiments show that there is no single temporal encoder that is best for every dataset. TCN performed best on WISDM and PAMAP2, while LSTM performed best overall on MHEALTH and HAR. The visualization and signal diagnostics suggest a useful interpretation. TCN is stronger when the discriminative pattern is local, bursty, and high-frequency, LSTM is stronger when the dataset benefits from smoother sequential evolution or longer temporal persistence.

Temporal SCMVC encoder comparison

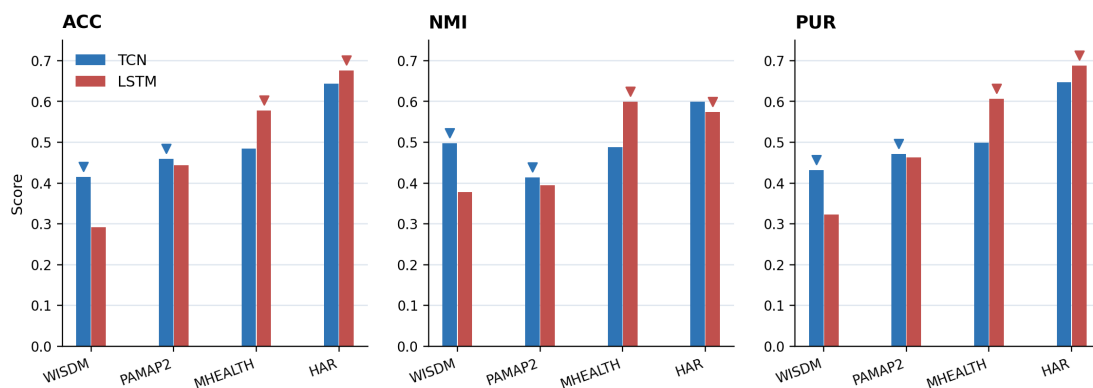


Figure 1. Main TCN vs LSTM comparison under the standardized 5,000-window setup.

2. Project Scope and Starting Point

The initial task is to become familiar with the paper '**Self-Weighted Contrastive Fusion for Deep Multi-View Clustering**' and its public SCMVC implementation. SCMVC is a deep multi-view clustering method that learns per-view representations, a fused global representation, and view-specific common representations. Its contrastive objective aligns the global representation with each view while allowing the model to weight views automatically.

After reproducing the repository and running the method on several available multi-view datasets, the project moved to time-series datasets from the UCI repository: HAR, WISDM, PAMAP2, and MHEALTH. The supervisor's final direction is to adapt SCMVC so that it works better for time-series data while keeping the modifications simple and interpretable.

- Stage 1: run and understand the original SCMVC repository.
- Stage 2: construct multi-view time-series datasets and evaluate SCMVC-style clustering on them.
- Stage 3: adapt SCMVC with temporal encoders, temporal pooling, and optional local window-level contrastive loss.
- Final task: add visual analysis explaining when TCN or LSTM is preferable, and prepare the final report and talk.

3. From Original SCMVC to Temporal SCMVC

The original SCMVC encoder expects each view to be a fixed vector. For time-series data, flattening a window too early can destroy temporal structure. The temporal adaptation keeps each view as a tensor with shape [samples, time, features] and lets the encoder learn from temporal order before pooling.

Original SCMVC	Temporal SCMVC extension
Input view is an already-vectorized feature matrix.	Input view is a temporal window [time, features].
Per-view encoder is a feed-forward vector encoder.	Per-view encoder is a TCN or LSTM temporal encoder.
Temporal order is not explicitly modeled.	Temporal order is modeled before pooling.

Fusion operates on per-view latent vectors.	Fusion still operates on latent vectors, preserving SCMVC's fusion idea.
Contrastive loss is global.	Global loss is preserved; local window-level loss is optionally added.

3.1 Implemented Code Components

Component	Files	Role
TCN temporal SCMVC	temporal_scmvc/ train_temporal_tcn.py, network_temporal_tcn.py	Main temporal convolutional version.
LSTM temporal SCMVC	temporal_scmvc/ train_temporal_lstm.py, network_temporal_lstm.py	Recurrent temporal encoder version.
Dataset loaders	dataloader_har_temporal.py, dataloader_wisdm_temporal.py, dataloader_mhealth_temporal.py, dataloader_pamap_temporal.py	Convert raw files into aligned multi-view temporal windows.
Sampling	temporal_scmvc/sampling.py	Fixed-seed stratified sampling for manageable and repeatable runs.
Visual analysis	temporal_scmvc/ create_pl_visualizations.py	Generates result, ablation, and signal-pattern figures.

3.2 Training Objective

The training pipeline keeps the two-stage SCMVC structure. First, each temporal autoencoder is pretrained using reconstruction loss. Then, contrastive training combines reconstruction loss with the original SCMVC global contrastive objective. The optional local loss splits each time window into shorter chunks and applies the contrastive objective at the chunk level.

Final objective with local loss enabled

Loss = reconstruction loss + global SCMVC contrastive loss + local_loss_weight * local window-level contrastive loss.

The local loss is disabled by default. In the final experiments, it is tested with **local_loss_weight = 0.1** and dataset-appropriate chunk sizes, but it did not improve performance.

4. Datasets and View Construction

Each dataset was transformed into a multi-view temporal clustering problem. A view corresponds either to a sensor group, a device/sensor modality, or a raw signal group. All views are synchronized at the window level before being passed into temporal SCMVC.

Dataset	Classes	Window	Views	View construction
WISDM	18	200	4	phone accelerometer, phone gyroscope, watch accelerometer, watch gyroscope
PAMAP2	12	50	4	heart rate, hand IMU, chest IMU, ankle IMU
MHEALTH	12	50	3	chest sensors, wrist/arm sensors, ankle sensors
HAR	6	128	3	raw inertial signal groups: total acceleration, body acceleration, body gyroscope

5. Experimental Protocol

The full raw datasets were too heavy for repeated local experiments on a MacBook Air, especially WISDM and PAMAP2. Therefore, the final comparison uses stratified subsets of 5,000 temporal windows. This is a hardware limitation, but the TCN/LSTM comparison remains fair because both encoders use the same selected samples, same seed, same view construction, and same main hyperparameters.

Setting	Value
Sample cap	5,000 temporal windows
Sampling	Stratified, seed 42
Main feature dimension	128
High-level feature dimension	64
Batch size	128
Temperature	0.5
Pretraining	30 epochs
Contrastive training	50 epochs
Evaluation	k-means on fused global representation H
Metrics	ACC, NMI, PUR

6. Main Results

The primary result is the standardized TCN vs LSTM comparison. Scores are reported as **ACC / NMI / PUR**.

Dataset	TCN ACC/NMI/PUR	LSTM ACC/NMI/PUR	Best encoder	Main interpretation
WISDM	0.4154 / 0.4977 / 0.4316	0.2918 / 0.3784 / 0.3232	TCN	TCN captures local motion patterns better.
PAMAP2	0.4596 / 0.4144 / 0.4714	0.4438 / 0.3948 / 0.4626	TCN	TCN is slightly but consistently stronger.
MHEALTH	0.4842 / 0.4884 / 0.4984	0.5776 / 0.5993 / 0.6062	LSTM	LSTM captures smoother temporal evolution better.
HAR	0.6432 / 0.5991 / 0.6474	0.6758 / 0.5740 / 0.6884	LSTM overall	LSTM wins ACC/PUR; TCN wins NMI.

Main result

The temporal SCMVC extension is general, but the best temporal encoder depends on the signal pattern. TCN is best for WISDM and PAMAP2, LSTM is best overall for MHEALTH and HAR.

6.1 View and Local-Loss Ablations

The ablation experiments show two useful effects. First, adding the fourth view helps TCN on both WISDM and PAMAP2. Second, the local window-level contrastive loss did not improve the tested datasets, it reduced performance compared with the global-only TCN.

Ablation results

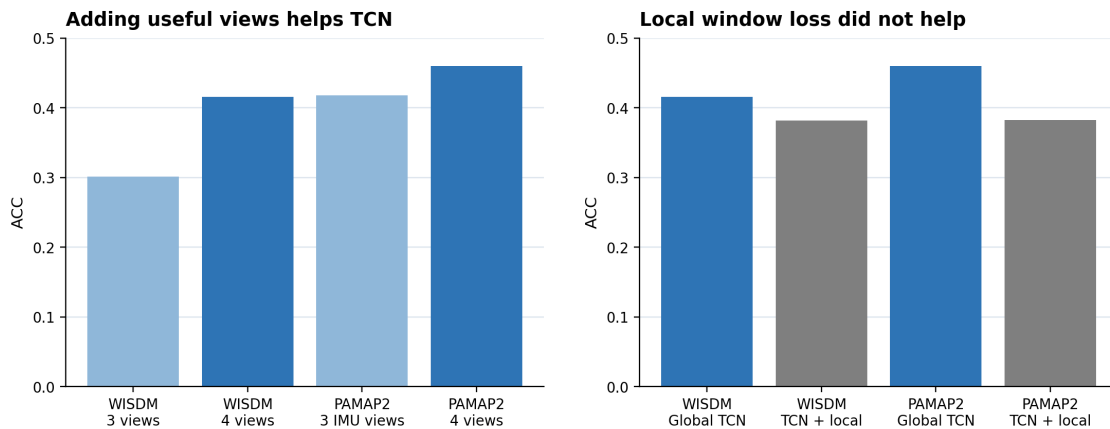


Figure 2. View-count and local-loss ablations using ACC.

Dataset	Global-only TCN	TCN + local loss	Effect
WISDM	0.4154 / 0.4977 / 0.4316	0.3820 / 0.4671 / 0.3920	Worse
PAMAP2	0.4596 / 0.4144 / 0.4714	0.3826 / 0.3024 / 0.4160	Worse

7. Visual Explanation: When Is TCN or LSTM Better?

To make the architecture behavior clearer, I added diagnostic visualizations based on the raw temporal windows. The goal is not to prove a theorem, but to connect observed clustering results with measurable time-series patterns.

Diagnostic	Meaning	Architectural implication
Local variation	Mean absolute change between consecutive time steps.	High values favor local temporal filters such as TCN.
High-frequency ratio	Share of FFT energy in higher frequencies.	High values indicate local oscillations or short motifs.
Burstiness	95th percentile first-difference magnitude divided by mean first-difference magnitude, capped for stability.	High values indicate sharp local events.
Local motif index	local variation x (1 + high-frequency ratio) x burstiness.	Higher values suggest TCN should be competitive.
Persistence index	autocorrelation half-life x (1 + long-lag absolute autocorrelation).	Higher values suggest longer sequential context, where LSTM can help.

Time-series diagnostics from raw temporal windows

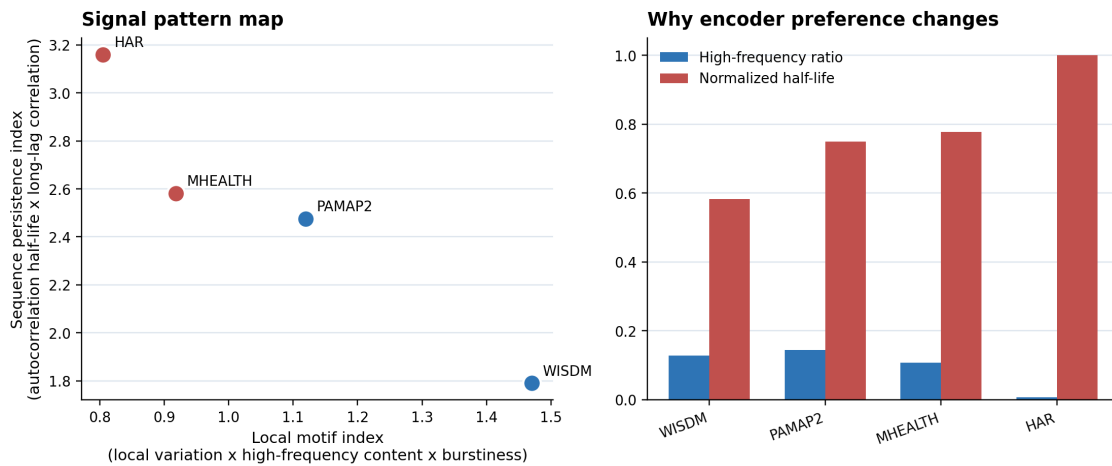


Figure 3. Diagnostic map, WISDM has the highest local motif index, while MHEALTH and HAR show more persistence.

Architecture choice deduced from signal pattern

Dataset	Best encoder	Local motif index	Persistence index	Pattern interpretation
WISDM	TCN	1.47	1.79	local / bursty motifs
PAMAP2	TCN	1.12	2.48	local / bursty motifs
MHEALTH	LSTM	0.92	2.58	smoother temporal evolution
HAR	LSTM overall	0.81	3.16	longer raw signal context; TCN still strong for NMI

Figure 4. Compact deduction of encoder preference from signal-pattern diagnostics.

WISDM has the highest local motif index and relatively low persistence, which matches the strong TCN advantage. PAMAP2 also has a high local motif score and high-frequency content, so TCN remains slightly better. MHEALTH has lower local motif strength and higher persistence, matching the LSTM advantage. HAR is more mixed LSTM wins ACC and PUR, while TCN wins NMI, which suggests that both local and longer temporal structure are relevant.

Representative temporal patterns by dataset



Figure 5. Representative standardized windows used for qualitative inspection of the temporal patterns.

8. Relation to Previous Reports

Earlier reports included the original SCMVC reproduction and static or preliminary temporal experiments. Some previous results are not directly comparable to the final standardized temporal runs because feature representation, sample size, and view construction changed. The most reliable comparison inside this final stage is therefore the standardized TCN vs LSTM comparison above.

Dataset	Earlier reference	Best current temporal result	Interpretation
WISDM	Static best: 0.3667 / 0.5042 / 0.3706	TCN: 0.4154 / 0.4977 / 0.4316	ACC and PUR improve NMI is slightly lower.
PAMAP2	Static best: 0.4149 / 0.3801 / 0.4841	TCN: 0.4596 / 0.4144 / 0.4714	ACC and NMI improve PUR is slightly lower.
MHEALTH	Earlier temporal LSTM: 0.6434 / 0.7044 / 0.6676	Current LSTM: 0.5776 / 0.5993 / 0.6062	Current general setup beats TCN but is below earlier configuration.
HAR	Static engineered features: 0.7884 / 0.6953 / 0.7884	LSTM raw signals: 0.6758 / 0.5740 / 0.6884	Not directly comparable because raw sequences replaced engineered features.

9. Limitations

- The final experiments use 5,000-window stratified subsets due to local hardware limitations.
- The results are single-run results, multi-seed averages would make the conclusions stronger.
- Local contrastive loss is tested with one weight and one chunk-size setting per dataset.
- The HAR temporal results use raw inertial signals, while the earlier static HAR baseline used engineered features.
- The time-series diagnostics are explanatory indicators, not formal guarantees of encoder performance.
- Stochastic time-series augmentations were not included in the final standardized comparison to avoid mixing too many changes at once.

10. Conclusions and Final Answer to the Research Question

The project shows that SCMVC can be adapted into a general time-series clustering framework by replacing vector encoders with temporal encoders and delaying pooling until after temporal feature extraction. The adaptation keeps the core SCMVC idea intact per-view learning, fused global representation, and contrastive alignment between global and view-specific common information.

The final answer is that architecture choice should depend on the time-series pattern. TCN is better when the dataset contains discriminative local motifs, abrupt changes, and high-frequency movement patterns, as observed for WISDM and PAMAP2. LSTM is better when class identity depends more on smoother temporal evolution and longer persistence, as observed for MHEALTH and mostly for HAR.

Final takeaway

The best temporal SCMVC variant is not universal. A practical rule is to start with TCN for local, bursty sensor patterns and with LSTM for smoother or more persistent temporal dynamics, then verify with the same clustering protocol.

References

- [1] S. Wu, Y. Zheng, Y. Ren, J. He, X. Pu, S. Huang, Z. Hao, and L. He, “Self-Weighted Contrastive Fusion for Deep Multi-View Clustering,” *IEEE Transactions on Multimedia*, vol. 26, pp. 9150-9162, 2024, doi: 10.1109/TMM.2024.3387298.
- [2] S. Wu, “SCMVC: Official code for Self-Weighted Contrastive Fusion for Deep Multi-View Clustering,” GitHub, 2024. [Online]. Available: <https://github.com/SongwuJob/SCMVC>
- [3] S. Bai, J. Z. Kolter, and V. Koltun, “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling,” *arXiv preprint arXiv:1803.01271*, 2018.
- [4] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
- [5] J. Reyes-Ortiz, D. Anguita, A. Ghio, L. Oneto, and X. Parra, “Human Activity Recognition Using Smartphones,” UCI Machine Learning Repository, 2013, doi: 10.24432/C54S4K.
- [6] A. Reiss, “PAMAP2 Physical Activity Monitoring,” UCI Machine Learning Repository, 2012, doi: 10.24432/C5NW2H.
- [7] O. Banos, R. Garcia, and A. Saez, “MHEALTH,” UCI Machine Learning Repository, 2014, doi: 10.24432/C5TW22.
- [8] G. Weiss, “WISDM Smartphone and Smartwatch Activity and Biometrics Dataset,” UCI Machine Learning Repository, 2019, doi: 10.24432/C5HK59.

You can cite them in text like: SCMVC [1], official code [2], TCN [3], LSTM [4], and datasets [5]-[8].

Appendix A: Representative Commands

Experiment	Command
WISDM TCN	<code>python train_temporal_tcn.py --dataset wisdm --wisdm_max_files_per_view 20 --max_samples 5000 --sample_strategy stratified --feature_dim 128 --high_feature_dim 64 --batch_size 128 --temperature 0.5 --pre_epochs 30 --con_epochs 50</code>
PAMAP2 TCN	<code>python train_temporal_tcn.py --dataset pamap2 --max_samples 5000 --sample_strategy stratified --feature_dim 128 --high_feature_dim 64 --batch_size 128 --temperature 0.5 --pre_epochs 30 --con_epochs 50</code>
MHEALTH LSTM	<code>python train_temporal_lstm.py --dataset mhealth --max_samples 5000 --sample_strategy stratified --feature_dim 128 --high_feature_dim 64 --batch_size 128 --temperature 0.5 --pre_epochs 30 --con_epochs 50</code>
HAR LSTM	<code>python train_temporal_lstm.py --dataset har --max_samples 5000 --sample_strategy stratified --feature_dim 128 --high_feature_dim 64 --batch_size 128 --temperature 0.5 --pre_epochs 30 --con_epochs 50</code>
Visualization	<code>python create_p1_visualizations.py</code>

Appendix B: Generated Artifacts

Artifact	Location
Final report	P1_Final_Comprehensive_Report_Fahad_Ali
Previous temporal progress report	Temporal_SCMVC_General_Method_Report_Fahad_Ali
Final figures	figures/p1_final/*.png
Visualization diagnostics	figures/p1_final/signal_diagnostics.csv
Temporal code	temporal_scmvc/